

Building ML Infrastructure

Samenvatting – MLOps4ECM

Doel van de handleiding

Deze handleiding beschrijft hoe machine learning-projecten geëvolueren van **experimentele notebooks** naar **productie-klare systemen**.

De focus ligt op **betrouwbaarheid, schaalbaarheid, reproduceerbaarheid en automatisatie** via MLOps-principes.

Machine learning in productie vereist meer dan alleen een goed model:

- gestructureerde code
 - robuuste infrastructuur
 - geautomatiseerde workflows
 - schaalbare opslag en deployment
-

Van notebooks naar productiecode

Probleem met notebooks

Jupyter notebooks zijn ideaal voor experimenten, maar ongeschikt voor productie:

- Slecht versiebeheer (JSON-formaat)
- Moeilijk te testen
- Geen duidelijke scheiding van verantwoordelijkheden
- Slechte integratie met CI/CD

Oplossing

- Verplaats logica naar **modulaire Python-bestanden**
- Gebruik notebooks enkel voor:
 - exploratie
 - visualisatie
- Voorzie:
 - herbruikbare functies
 - unit tests
 - duidelijke projectstructuur

Resultaat: onderhoudbare, testbare en schaalbare ML-code.

Model deployment en serving

Na training moet een model beschikbaar zijn voor andere systemen.

Deploymentstrategieën

- **Server-side deployment**
 - model draait op een centrale server
 - clients vragen voorspellingen op
- **Edge deployment**
 - model draait lokaal (bv. IoT, smartphone)
 - lage latency en privacyvoordelen

In de cursus ligt de focus op **server-side deployment**.

Communicatieprotocollen voor ML-services

REST API

- HTTP-gebaseerd (GET, POST, ...)
- Breed ondersteund
- Eenvoudig en schaalbaar
- Iets hogere latency

WebSockets

- Permanente verbinding
- Realtime communicatie
- Geschikt voor dashboards
- Complexer om te schalen

MQTT

- Lichtgewicht protocol
- Ideaal voor IoT en sensoren
- Minder browservriendelijk

FastAPI voor model serving

Waarom FastAPI?

- Modern Python webframework
- Hoge performance
- Type hints en automatische validatie
- Automatische API-documentatie (Swagger UI)

Gebruik

- ML-model laden bij startup
- `/predict` endpoint aanbieden
- JSON in → voorspelling uit

FastAPI is ideaal voor het bouwen van **productie-waardige ML API's**.

Frontends voor ML-API's

Een API heeft clients nodig om ermee te communiceren.

Mogelijke frontends

- **JavaScript frameworks** (React, Vue, Svelte)
- **Python-webapps** (Flask)
- **Dashboards** (Streamlit, Dash, Gradio)
- Mobiele, desktop of embedded apps

Keuze hangt af van

- doelgroep
 - schaal
 - snelheid van ontwikkeling
 - technische vaardigheden van het team
-

Containers en virtuele machines

Virtuele Machines (VMs)

- Volledige OS per applicatie
- Zware overhead
- Trage opstarttijd

Containers

- Delen de kernel van het host-systeem
- Lichtgewicht en snel
- Ideaal voor ML-workloads
- Reproduceerbare omgevingen

Containers zijn de standaard voor ML-deployment.

Docker voor Machine Learning

Waarom Docker?

- Lost "it works on my machine" op
- Combineert:
 - code
 - dependencies
 - runtime-omgeving

Best practices

- Gebruik vaste versies (geen `latest`)
- Definieer dependencies in `requirements.txt`

- Versiebeheer container images

Docker vormt de **basis voor schaalbare ML-systemen**.

Container orchestration

Docker Compose

- Geschikt voor lokale omgevingen
- Beheert meerdere containers tegelijk
- Eén commando om alles te starten

Kubernetes

- Industriestandaard voor productie
- Schaalbaarheid, failover, rolling updates
- Complexer, maar zeer krachtig

Samenvatting

- Compose → ontwikkeling & demo's
 - Kubernetes → productie & schaal
-

Infrastructuurbeheer

Cloud vs On-premise

- **Cloud**: flexibel, schaalbaar, pay-as-you-go
- **On-premise**: meer controle, geschikt voor gevoelige data

Cloud lagen

- **IaaS**: virtuele machines
 - **PaaS**: beheerde platformen
 - **SaaS**: kant-en-klare AI-services
-

Infrastructure as Code (IaC)

Infrastructuur wordt beschreven met code:

- versiebeheer
- herhaalbaarheid
- automatisatie

Tools

- Terraform
- Kubernetes YAML
- GitOps tools (Argo CD, Flux)

laC voorkomt "snowflake servers" en manuele fouten.

Scalable storage voor ML

Waarom Git niet volstaat

- Grote binaire bestanden
- Datasets veranderen vaak
- Slechte prestaties bij grote files

Object storage

- Opslag via API (bv. S3)
- Geschikt voor:
 - datasets
 - modellen
 - logs

S3-compatibele oplossingen

- Cloud: AWS S3, Azure Blob, GCS
- On-prem: MinIO, Ceph

Object storage vormt de **ruggengraat van ML-data**.

CI/CD voor Machine Learning

Waarom automatiseren?

- Minder fouten
- Snellere iteratie
- Betere reproduceerbaarheid

CI

- Automatisch testen bij codewijzigingen
- Voorkomt defecte code in productie

CD

- Automatische deployment na tests
- Continue levering van verbeteringen

GitHub Actions

- CI/CD direct in GitHub
- Geschikt voor:
 - testen
 - trainen

- container builds
- deployment
- artifact uploads

Data pipelines & orchestration

CI/CD $\neq$ data orchestration

- CI/CD $\rightarrow$ code-gedreven
- Orchestration $\rightarrow$ data- en tijd-gedreven

DAGs (Directed Acyclic Graphs)

- Taken met afhankelijkheden
- Parallelisatie
- Herstartbare pipelines

Tools

- **Airflow**: stabiel, maar boilerplate
- **Dagster**: asset-gericht
- **Prefect**: flexibel en Pythonic

Conclusie

Machine learning in productie vraagt:

- software-engineering discipline
- automatisatie
- infrastructuurkennis

Je hoeft niet alles meteen te gebruiken:

Start eenvoudig en schaal mee met je project.

Containers, object storage, CI/CD en data pipelines vormen samen de **fundering van robuuste ML-systemen**.